PYTHON STANDARD LIBRARY

Six Essential Modules

An in-depth, practical guide to csv, json, random, time, datetime, and pathlib — core concepts, key functions, worked examples, and common gotchas.

csv json random time datetime pathlib

csv

`import csv`

The **csv** module reads and writes CSV (comma-separated values) files — the plain-text spreadsheet format used everywhere for tabular data exports. It correctly handles quoted fields, embedded commas, embedded newlines, and different delimiters, which is much safer than manually splitting lines on a comma.

CORE CONCEPTS

A CSV file is just text: each line is a row, and commas separate columns. The catch is that a field might itself contain a comma or a newline, so it gets wrapped in quotes (e.g. **"Smith, John"**). The csv module parses these rules for you so you never have to write fragile string-splitting code. It reads/writes using "dialects" — presets for delimiter, quote character, and line endings (the default is excel-style CSV).

KEY FUNCTIONS & CLASSES

`csv.reader(f, delimiter=',')`	Reads rows as lists of strings; delimiter can be changed (e.g. '\t' for TSV).
`csv.writer(f)`	Writes rows (lists/tuples) to a file as CSV text.
`csv.DictReader(f)`	Reads rows as ordered dictionaries, using the header row as keys.
`csv.DictWriter(f, fieldnames)`	Writes dictionaries as CSV rows in a defined column order.
`writer.writerow(row)`	Writes a single row.
`writer.writerows(rows)`	Writes multiple rows at once from a list of lists.
`csv.Sniffer()`	Auto-detects the dialect (delimiter, quoting style) of an unknown CSV file.
`quoting=csv.QUOTE_ALL`	Option to force quotes around every field when writing.

READING & WRITING

```python
# reading with DictReader
import csv

with open("people.csv", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["age"])
```

```python
# writing with DictWriter
import csv

rows = [{"name": "Alice", "age": 30},
        {"name": "Bob", "age": 25}]

with open("out.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "age"])
    w.writeheader()
    w.writerows(rows)
```

CUSTOM DELIMITERS (TSV EXAMPLE)

```python
import csv

with open("data.tsv", newline="") as f:
    reader = csv.reader(f, delimiter="\t")
    for row in reader:
        print(row)
```

Gotcha: Always open files with **newline=""** when reading/writing CSV on Windows — otherwise extra blank lines can appear between rows because of how the csv module handles line endings itself.

Use it for: importing/exporting spreadsheet data, logging structured records, ETL pipelines, or moving data between tools like Excel, Google Sheets, and Python.

{ } json

import json

The **json** module converts between Python objects (dicts, lists, strings, numbers, booleans) and JSON text — the standard format for web APIs, config files, and data interchange. JSON maps closely to Python's own data structures, which makes this module very lightweight to use.

CORE CONCEPTS

"Serializing" (or "dumping") means turning a Python object into a JSON string; "deserializing" (or "loading") means turning JSON text back into Python objects. Type mapping is mostly 1-to-1: dict ↔ object, list ↔ array, str ↔ string, int/float ↔ number, True/False ↔ true/false, None ↔ null. Not every Python type is JSON-serializable by default (e.g. datetime, sets) — you need custom encoders for those.

KEY FUNCTIONS

`json.dumps(obj)`	Converts a Python object to a JSON-formatted string.
`json.loads(s)`	Parses a JSON string into a Python object.
`json.dump(obj, f)`	Writes a Python object as JSON directly to a file.
`json.load(f)`	Reads JSON from a file and parses it into a Python object.
`indent=N`	Pretty-prints output with N spaces of indentation.
`sort_keys=True`	Outputs dictionary keys in sorted order (useful for diffs/tests).
`default=fn`	Custom function to serialize otherwise non-JSON-safe types.
`json.JSONDecodeError`	Exception raised when parsing invalid JSON text.

BASIC SERIALIZATION

```python
# Python -> JSON
import json

data = {"name": "Alice", "age": 30,
        "tags": ["a", "b"]}

text = json.dumps(data, indent=2)
print(text)
```

```python
# JSON -> Python
import json

text = '{"name": "Alice", "age": 30}'
parsed = json.loads(text)
print(parsed["name"])  # Alice
```

FILES & CUSTOM TYPES

```python
import json
from datetime import datetime

# write / read JSON files
with open("data.json", "w") as f:
    json.dump({"created": datetime.now()}, f,
              default=lambda o: o.isoformat())

with open("data.json") as f:
    loaded = json.load(f)
```

Gotcha: json.dumps() raises **TypeError** on objects it doesn't know how to serialize (like datetime or a custom class) — use the **default=** parameter or convert the value manually first.

Use it for: talking to web APIs, saving config or settings files, caching structured data, and exchanging data between services in a human-readable format.

random

`import random`

The **random** module generates pseudo-random numbers and makes random selections — useful for simulations, games, sampling, testing, and shuffling data. It's "pseudo-random" because the numbers come from a deterministic algorithm seeded by a starting value.

CORE CONCEPTS

Every run uses an internal generator (Mersenne Twister) seeded automatically from system entropy, so results differ each run by default. Call **random.seed(n)** to make results reproducible — useful for testing or demos. This module is **not cryptographically secure**; for security-sensitive randomness (tokens, passwords) use the **secrets** module instead.

KEY FUNCTIONS

`random.seed(n)`	Sets the starting seed so results become reproducible.
`random.random()`	Returns a random float between 0.0 and 1.0.
`random.uniform(a, b)`	Returns a random float between a and b.
`random.randint(a, b)`	Returns a random integer between a and b (inclusive).
`random.randrange(start, stop, step)`	Like range(), but returns one random value from it.
`random.choice(seq)`	Returns a single random element from a sequence.
`random.choices(seq, k=n, weights=w)`	Returns n random elements, with replacement; supports weighting.
`random.sample(seq, k)`	Returns k unique random elements, without replacement.
`random.shuffle(list)`	Shuffles a list in place, randomizing its order.
`random.gauss(mu, sigma)`	Returns a value from a normal (Gaussian) distribution.

EXAMPLES

```python
# basic selection & shuffling
import random

dice_roll = random.randint(1, 6)
winner = random.choice(["Alice", "Bob"])

cards = ["A", "K", "Q", "J"]
random.shuffle(cards)
```

```python
# sampling & weighted choice
import random

lotto = random.sample(range(1, 50), 6)

pick = random.choices(
    ["common", "rare"],
    weights=[90, 10], k=1
)
```

REPRODUCIBLE RESULTS

```python
import random

random.seed(42)
print(random.randint(1, 100))  # same output every run
```

Gotcha: Never use the **random** module for passwords, tokens, or anything security-related — it's predictable if the seed is known. Use **secrets.token_hex()** or **secrets.choice()** instead.

Use it for: games, shuffling quiz questions, randomized test data, Monte Carlo simulations, and A/B test assignment.

time

```
import time
```

The **time** module works with low-level time functions — measuring elapsed time, pausing execution, and reading the system clock. It's the older, closer-to-the-OS counterpart to the higher-level **datetime** module.

CORE CONCEPTS

Most of this module revolves around the Unix "epoch" — the number of seconds since January 1, 1970 UTC. **time.time()** gives you that number, which is easy to store or compare but not human-readable on its own. For accurate benchmarking, use **time.perf_counter()** rather than **time.time()**, since it isn't affected by system clock adjustments.

KEY FUNCTIONS

`time.time()`	Returns the current time in seconds since the Unix epoch.
`time.sleep(s)`	Pauses program execution for s seconds.
`time.perf_counter()`	High-precision timer, ideal for measuring code duration (not wall-clock accurate).
`time.monotonic()`	A clock that never goes backwards; good for measuring intervals safely.
`time.localtime()`	Converts epoch seconds into a struct_time in local time.
`time.gmtime()`	Converts epoch seconds into a struct_time in UTC.
`time.strftime(fmt, t)`	Formats a struct_time (or current time) as a string.
`time.strptime(s, fmt)`	Parses a string into a struct_time object.

TIMING CODE

```python
# benchmarking a block
import time

start = time.perf_counter()
time.sleep(1)
elapsed = time.perf_counter() - start

print(f"Took {elapsed:.2f}s")
```

```python
# formatting the clock
import time

now = time.localtime()
print(time.strftime(
    "%Y-%m-%d %H:%M:%S", now
))
```

EPOCH TIMESTAMPS

```python
import time

ts = time.time()          # e.g. 1799472000.123456
readable = time.ctime(ts) # human readable string
print(readable)
```

Gotcha: time.time() can jump backward or forward if the system clock is adjusted (NTP sync, manual change). For measuring durations, prefer **time.perf_counter()** or **time.monotonic()** instead.

Use it for: benchmarking code, adding delays/retries, rate-limiting, and low-level timestamping at the system level.

datetime

`import datetime`

The **datetime** module provides classes for working with calendar dates and times — creating, formatting, comparing, and doing arithmetic on dates in a much more readable, structured way than the raw **time** module.

CORE CONCEPTS

Four main classes matter: **date** (year/month/day only), **time** (hour/minute/second only), **datetime** (both combined), and **timedelta** (a duration used for arithmetic between dates). Datetimes can be "naive" (no timezone info) or "aware" (carry a timezone) — mixing the two in comparisons raises an error, so pick one approach per project.

KEY FUNCTIONS & CLASSES

`datetime.now()`	Returns the current local date and time (naive).
`datetime.now(tz)`	Returns the current date and time in a given timezone (aware).
`datetime.date(y, m, d)`	Creates a date object for a specific calendar day.
`datetime.strftime(fmt)`	Formats a datetime object as a string.
`datetime.strptime(s, fmt)`	Parses a string into a datetime object.
`datetime.fromisoformat(s)`	Parses an ISO 8601 string (e.g. "2026-09-09") directly.
`timedelta(days=, hours=)`	Represents a duration, used for date/time arithmetic.
`dt.weekday() / isoweekday()`	Returns the day of the week as an integer.
`dt.replace(...)`	Returns a new datetime with specific fields changed.

ARITHMETIC & FORMATTING

```python
# date arithmetic
from datetime import datetime, timedelta

now = datetime.now()
next_week = now + timedelta(days=7)
diff = next_week - now
print(diff.days)  # 7
```

```python
# formatting & parsing
from datetime import datetime

now = datetime.now()
text = now.strftime("%B %d, %Y")

parsed = datetime.strptime(
    "2026-09-09", "%Y-%m-%d")
```

COMMON FORMAT CODES

`%Y / %m / %d`	4-digit year / 2-digit month / 2-digit day.
`%H / %M / %S`	24-hour, minute, second.
`%B / %A`	Full month name / full weekday name.

Gotcha: Comparing a naive datetime to an aware one raises **TypeError**. Be consistent — if you use timezones anywhere, use **datetime.now(timezone.utc)** everywhere in that project.

Use it for: scheduling, age/duration calculations, formatting human-readable dates, and parsing date strings from files, logs, or APIs.

pathlib

`from pathlib import Path`

The **pathlib** module offers an object-oriented way to handle filesystem paths — a modern, cross-platform replacement for string-based paths and the older **os.path** module. Paths become objects with useful methods instead of plain strings you manipulate manually.

CORE CONCEPTS

Path objects support the "/" operator to join path segments, automatically using the correct separator for the current OS (backslash on Windows, forward slash elsewhere). This avoids bugs from hardcoding "/" or "\\" and makes path-building code far more readable than **os.path.join()** chains.

KEY FUNCTIONS & METHODS

`Path("x")`	Creates a path object representing a file or folder.
`Path.cwd()`	Returns the current working directory as a Path.
`Path.home()`	Returns the user's home directory as a Path.
`Path.exists()`	Checks whether the path exists on disk.
`Path.is_file() / is_dir()`	Checks whether the path is a file or a directory.
`Path.mkdir(parents=True)`	Creates a new directory, optionally creating parents too.
`Path.glob(pattern)`	Finds files matching a pattern, e.g. "*.txt".
`Path.rglob(pattern)`	Like glob, but searches recursively through subfolders.
`Path.read_text() / write_text()`	Reads or writes a file's text content directly.
`Path.suffix / .stem / .name`	Extension, filename without extension, and full filename.
`Path.resolve()`	Returns the absolute, fully-resolved version of the path.

BUILDING & INSPECTING PATHS

```python
# building paths
from pathlib import Path

folder = Path("documents")
folder.mkdir(exist_ok=True)

file = folder / "notes.txt"
file.write_text("Hello, pathlib!")
```

```python
# inspecting a path
from pathlib import Path

p = Path("documents/notes.txt")
print(p.name)    # notes.txt
print(p.stem)    # notes
print(p.suffix)  # .txt
print(p.parent)  # documents
```

SEARCHING DIRECTORIES

```python
from pathlib import Path

folder = Path("documents")

# list all .txt files in this folder
for txt_file in folder.glob("*.txt"):
    print(txt_file.name)

# search recursively through all subfolders
for py_file in folder.rglob("*.py"):
    print(py_file)
```

Gotcha: Path objects aren't automatically strings. Some older libraries expect a plain string, so you may need to wrap it explicitly with **str(path)** when passing it to such functions.

Use it for: building, checking, and manipulating file paths in a readable, cross-platform way; walking directory trees; and reading/writing small files without manual open()/close() calls.